

Unit 1

Differences between C and C++:

C	C++
C was developed by Dennis Ritchie between the year 1969 and 1973 at AT&T Bell Labs.	C++ was developed by Bjarne Stroustrup in 1979.
C does not support polymorphism, encapsulation, and inheritance which means that C does not support object oriented programming.	C++ supports polymorphism, encapsulation, and inheritance because it is an object oriented programming language.
C is a subset of C++.	C++ is a superset of C.
C contains 32 keywords.	C++ contains 63 keywords.
For the development of code, C supports procedural programming.	C++ is known as hybrid language because C++ supports both procedural and object oriented programming paradigms.
Data and functions are separated in C because it is a procedural programming language.	Data and functions are encapsulated together in form of an object in C++.
C does not support information hiding.	Data is hidden by the Encapsulation to ensure that data structures and operators are used as intended.
Built-in data types is supported in C.	Built-in & user-defined data types is supported in C++.
Function and operator overloading is not supported in C.	Function and operator overloading is supported by C++.
Namespace features are not present inside the C.	Namespace is used by C++, which avoid name collisions.
Reference variables are not supported by C.	Reference variables are supported by C++.
Virtual and friend functions are not supported by C.	Virtual and friend functions are supported by C++.
C does not support inheritance.	C++ supports inheritance.
Direct support for exception handling is not supported by C.	Exception handling is supported by C++.

Object Oriented Programming Paradigm:

Object-oriented programming (OOP) is a programming paradigm based upon objects (having both data and methods) that aims to incorporate the advantages of modularity and reusability. Objects, which are usually instances of classes, are used to interact with one another to design applications and computer programs.

The important features of object-oriented programming are –

- Bottom-up approach in program design

- Programs organized around objects, grouped in classes
- Focus on data with methods to operate upon object's data
- Interaction between objects through functions
- Reusability of design through creation of new classes by adding features to existing classes

Basic concepts of object oriented programming:

Object oriented programming is a type of programming which uses objects and classes its functioning. The object oriented programming is based on real world entities like inheritance, polymorphism, data hiding, etc. It aims at binding together data and function work on these data sets into a single entity to restrict their usage.

Some basic concepts of object oriented programming are –

- CLASS
- OBJECTS
- ENCAPSULATION
- POLYMORPHISM
- INHERITANCE
- ABSTRACTION

Benefits of OOP:

1. Modularity for easier troubleshooting
2. Reuse of code through inheritance
3. Flexibility through polymorphism
4. Effective problem solving

Object Oriented Languages:

C++, Java, Python, C#, Perl, etc.

A Sample Program:

Refer the program below:

```
// Sample C++ Program
#include<iostream>
using namespace std;
int main()
{
    cout<<"Hello World!";
    return 0;
}
```

Ouput:

Hello World!

□ `// Sample C++ Program`

In C++, any line starting with `//` is a comment. Comments are intended for the person reading the code to better understand the functionality of the program. It is completely ignored by the C++ compiler.

□ `#include <iostream>`

The `#include` is a preprocessor directive used to include files in our program. The above code is including the contents of the **iostream** file.

This allows us to use `cout` in our program to print output on the screen.

For now, just remember that we need to use `#include <iostream>` to use `cout` that allows us to print output on the screen.

□ `int main() {...}`

A valid C++ program must have the `main()` function. The curly braces indicate the start and the end of the function.

The execution of code begins from this function.

□ `cout << "Hello World!";`

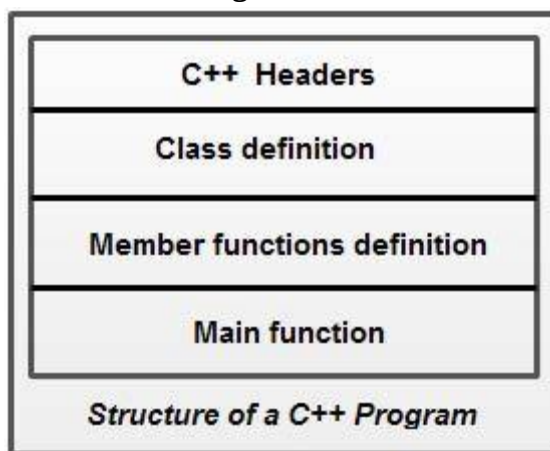
`cout` prints the content inside the quotation marks. It must be followed by `<<` followed by the format string. In our example, `"Hello World!"` is the format string.

Note: `;` is used to indicate the end of a statement.

□ `return 0;`

The `return 0;` statement is the **"Exit status"** of the program. In simple terms, the program ends with this statement.

Structure of C++ Program:



Introduction to OOP:

The origins of C++:

C++ was developed by Bjarne Stroustrup at Bell Laboratories over a period starting in 1979. Since C++ is an attempt to add object-oriented features (plus other improvements) to C, earlier it was called as “C with Objects”. As the language developed, Stroustrup named it as C++ in 1983. The name C++ suggests “C incremented” (recall the ++ is an increment operator of C).

C++ was made available outside Bell Laboratories in 1985. The first commercial C++ compiler, Cfront, was released in 1985. It was only a front-end compiler for C. The American National Standard Institute (ANSI) formed a committee for (precise description of computer language) C++, in 1989. The first draft standards were published in 1995.

What is Object Oriented Programming?

Object-oriented programming is a programming paradigm built on the concept of objects that contain both data and code to modify the data. Object-oriented programming mimics a lot of the real-world attributes of objects.

Some C++ Fundamentals:

Header Files:

Header files contain definitions of **Functions and Variables**, which is imported or used into any C++ program by using the pre-processor `#include` statement. Header file have an extension ".h" which contains C++ function declaration and macro definition. Each header file contains information (or declarations) for a particular group of functions. Like **stdio.h** header file contains declarations of standard input and output functions available in C++ which is used for get the input and print the output. Similarly, the header file **math.h** contains declarations of mathematical functions available in C++.

When we want to use any function in our C++ program then first we need to import their definition from C++ library, for importing their declaration and definition we need to include header file in program by using `#include`. Header file include at the top of any C++ program. For example if we use `clrscr()` in C++ program, then we need to include, `conio.h` header file, because in `conio.h` header file definition of `clrscr()` (for clear screen) is written in `conio.h` header file. Both user and system header files are include using the pre-processing directive `#include`.

Namespaces:

Namespaces allow us to group named entities that otherwise would have *global scope* into narrower scopes, giving them *namespace scope*. This allows organizing the elements of programs into different logical scopes referred to by names.

- Namespace is a feature added in C++ and not present in C.
- A namespace is a declarative region that provides a scope to the identifiers (names of the types, function, variables etc) inside it.
- Multiple namespace blocks with the same name are allowed. All declarations within those blocks are declared in the named scope.
 - Namespace declarations appear only at global scope.

- Namespace declarations can be nested within another namespace.
- Namespace declarations don't have access specifiers. (Public or private)
- No need to give semicolon after the closing brace of definition of namespace.
- We can split the definition of namespace over several units.

Introduction to C++ Classes and Objects:

A class is used to specify the form of an object and it combines data representation and methods for manipulating that data into one neat package. The data and functions within a class are called members of the class.

When you define a class, you define a blueprint for a data type. This doesn't actually define any data, but it does define what the class name means, that is, what an object of the class will consist of and what operations can be performed on such an object.

A class definition starts with the keyword **class** followed by the class name; and the class body, enclosed by a pair of curly braces. A class definition must be followed either by a semicolon or a list of declarations. For example, we defined the Box data type using the keyword **class** as follows –

```
class Box {
    public:
        double length; // Length of a box
        double breadth; // Breadth of a box
        double height; // Height of a box
};
```

The keyword **public** determines the access attributes of the members of the class that follows it. A public member can be accessed from outside the class anywhere within the scope of the class object. You can also specify the members of a class as **private** or **protected**.

Objects: A class provides the blueprints for objects, so basically an object is created from a class. We declare objects of a class with exactly the same sort of declaration that we declare variables of basic types. Following statements declare two objects of class Box –

```
Box Box1;    // Declare Box1 of type Box
Box Box2;    // Declare Box2 of type Box
```

Both of the objects Box1 and Box2 will have their own copy of data members.

Function Overloading, Operator Overloading:

C++ allows you to specify more than one definition for a **function** name or an **operator** in the same scope, which is called **function overloading** and **operator overloading** respectively.

An overloaded declaration is a declaration that is declared with the same name as a previously declared declaration in the same scope, except that both declarations have different arguments and obviously different definition (implementation).

When you call an overloaded **function** or **operator**, the compiler determines the most appropriate definition to use, by comparing the argument types you have used to call the

function or operator with the parameter types specified in the definitions. The process of selecting the most appropriate overloaded function or operator is called **overload resolution**.

Inheritance:

Inheritance allows us to define a class in terms of another class, which makes it easier to create and maintain an application. This also provides an opportunity to reuse the code functionality and fast implementation time.

When creating a class, instead of writing completely new data members and member functions, the programmer can designate that the new class should inherit the members of an existing class. This existing class is called the **base** class, and the new class is referred to as the **derived** class.

Constructors and Destructors:

A class constructor is a special member function of a class that is executed whenever we create new objects of that class.

A constructor will have exact same name as the class and it does not have any return type at all, not even void. Constructors can be very useful for setting initial values for certain member variables.

A default constructor does not have any parameter, but if you need, a constructor can have parameters. This helps you to assign initial value to an object at the time of its creation. A destructor is a special member function of a class that is executed whenever an object of it's class goes out of scope or whenever the delete expression is applied to a pointer to the object of that class.

A destructor will have exact same name as the class prefixed with a tilde (~) and it can neither return a value nor can it take any parameters. Destructor can be very useful for releasing resources before coming out of the program like closing files, releasing memories etc.

SAMPLE PROGRAMS:

1. Program to swap two numbers

```
#include <iostream>
using namespace std;
int main()
{
    int a = 5, b = 10, temp;
    cout << "Before swapping." << endl;
    cout << "a = " << a << ", b = " << b << endl;
    temp = a;
    a = b;
    b = temp;
    cout << "\nAfter swapping." << endl;
    cout << "a = " << a << ", b = " << b << endl;
    return 0;
}
```

2. Program to find ASCII value of a character

```
#include <iostream>
using namespace std;
int main()
{
    char c;
    cout << "Enter a character: ";
    cin >> c;
    cout << "ASCII Value of " << c << " is " << int(c);
    return 0;
}
```

3. Program to check whether the number is even or odd

```
#include <iostream>
using namespace std;
int main()
{
    int n;
    cout << "Enter an integer: ";
    cin >> n;
    if ( n % 2 == 0)
        cout << n << " is even.";
    else
        cout << n << " is odd.";
    return 0;
}
```

4. Program to find roots of quadratic equation

If determinant > 0,	$\text{root1} = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$ $\text{root2} = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$
If determinant = 0,	$\text{root1} = \text{root2} = \frac{-b}{2a}$
If determinant < 0,	$\text{root1} = \frac{-b}{2a} + i \frac{\sqrt{-(b^2 - 4ac)}}{2a}$ $\text{root2} = \frac{-b}{2a} - i \frac{\sqrt{-(b^2 - 4ac)}}{2a}$

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
```

```

float a, b, c, x1, x2, discriminant, realPart, imaginaryPart;
cout << "Enter coefficients a, b and c: ";
cin >> a >> b >> c;
discriminant = b*b - 4*a*c;
if (discriminant > 0) {
    x1 = (-b + sqrt(discriminant)) / (2*a);
    x2 = (-b - sqrt(discriminant)) / (2*a);
    cout << "Roots are real and different." << endl;
    cout << "x1 = " << x1 << endl;
    cout << "x2 = " << x2 << endl;
}

else if (discriminant == 0) {
    cout << "Roots are real and same." << endl;
    x1 = -b/(2*a);
    cout << "x1 = x2 = " << x1 << endl;
}

else {
    realPart = -b/(2*a);
    imaginaryPart = sqrt(-discriminant)/(2*a);
    cout << "Roots are complex and different." << endl;
    cout << "x1 = " << realPart << "+" << imaginaryPart << "i" << endl;
    cout << "x2 = " << realPart << "-" << imaginaryPart << "i" << endl;
}

return 0;
}

```

5. Program to reverse an integer

```

#include <iostream>
using namespace std;
int main() {
    int n, reversedNumber = 0, remainder;
    cout << "Enter an integer: ";
    cin >> n;
    while(n != 0) {
        remainder = n%10;
        reversedNumber = reversedNumber*10 + remainder;
        n /= 10;
    }
    cout << "Reversed Number = " << reversedNumber;
    return 0;
}

```